THEME DESCRIPTION

Students will be learning exploratory data analysis.

WEEKLY LEARNING GOAL (SUB-GOAL LEARNING)

CLO022, SUB-CLO-2: Students can identify, analyze and implement basic data analysis using Python and its Library.

SOFTWARE / HARDWARE REQUIREMENTS

1. Anaconda
2. Jupyter Notebook
3. Google Colab
4. Python 3.7 or Higher

PRACTICUM STEPS

A. GUIDED PRACTICUM

1. Importing Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

2. Check the sns version

```
print(sns.__version__)
```

```
0.13.2
```

3. Import 'sales.csv' data

```
mobil = pd.read_csv('sales.csv')
mobil.head(10)
```

Python

	Manufacturer	Model	Sales_in_thousands	__year_resale_value	Vehicle_type	Price_in_thousands	Engine_size
0	Acura	Integra	16.919	16.360	Passenger	21.50	1.8
1	Acura	TL	39.384	19.875	Passenger	28.40	3.2
2	Acura	CL	14.114	18.225	Passenger	NaN	3.2
3	Acura	RL	8.588	29.725	Passenger	42.00	3.5
4	Audi	A4	20.397	22.255	Passenger	23.99	1.8
5	Audi	A6	18.780	23.555	Passenger	33.95	2.8
6	Audi	A8	1.380	39.000	Passenger	62.00	4.2
7	BMW	323i	19.747	NaN	Passenger	26.99	2.5
8	BMW	328i	9.231	28.675	Passenger	33.40	2.8
9	BMW	528i	17.527	36.125	Passenger	38.90	2.8

4. See the mobil info

```
mobil.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 157 entries, 0 to 156
Data columns (total 16 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   Manufacturer         157 non-null    object 
 1   Model                157 non-null    object 
 2   Sales_in_thousands   157 non-null    float64
 3   __year_resale_value  121 non-null    float64
 4   Vehicle_type         157 non-null    object 
 5   Price_in_thousands   155 non-null    float64
 6   Engine_size          156 non-null    float64
 7   Horsepower           156 non-null    float64
 8   Wheelbase            156 non-null    float64
 9   Width                156 non-null    float64
 10  Length               156 non-null    float64
 11  Curb_weight          155 non-null    float64
 12  Fuel_capacity        156 non-null    float64
 13  Fuel_efficiency      154 non-null    float64
 14  Latest_Launch        157 non-null    object 
 15  Power_perf_factor    155 non-null    float64
dtypes: float64(12), object(4)
memory usage: 19.8+ KB
```

5. See the shape of dataframe mobil

```
mobil.shape
```

```
(157, 16)
```

6. See the columns in mobil

```
mobil.columns
```

```
Index(['Manufacturer', 'Model', 'Sales_in_thousands', '__year_resale_value',
       'Vehicle_type', 'Price_in_thousands', 'Engine_size', 'Horsepower',
       'Wheelbase', 'Width', 'Length', 'Curb_weight', 'Fuel_capacity',
       'Fuel_efficiency', 'Latest_Launch', 'Power_perf_factor'],
      dtype='object')
```

7. Make a Profile Report about mobil

```
from ydata_profiling import ProfileReport
# Generate the profile report
report = ProfileReport(mobil)

# Save the report to a file
report.to_file("data_profile_report.html")

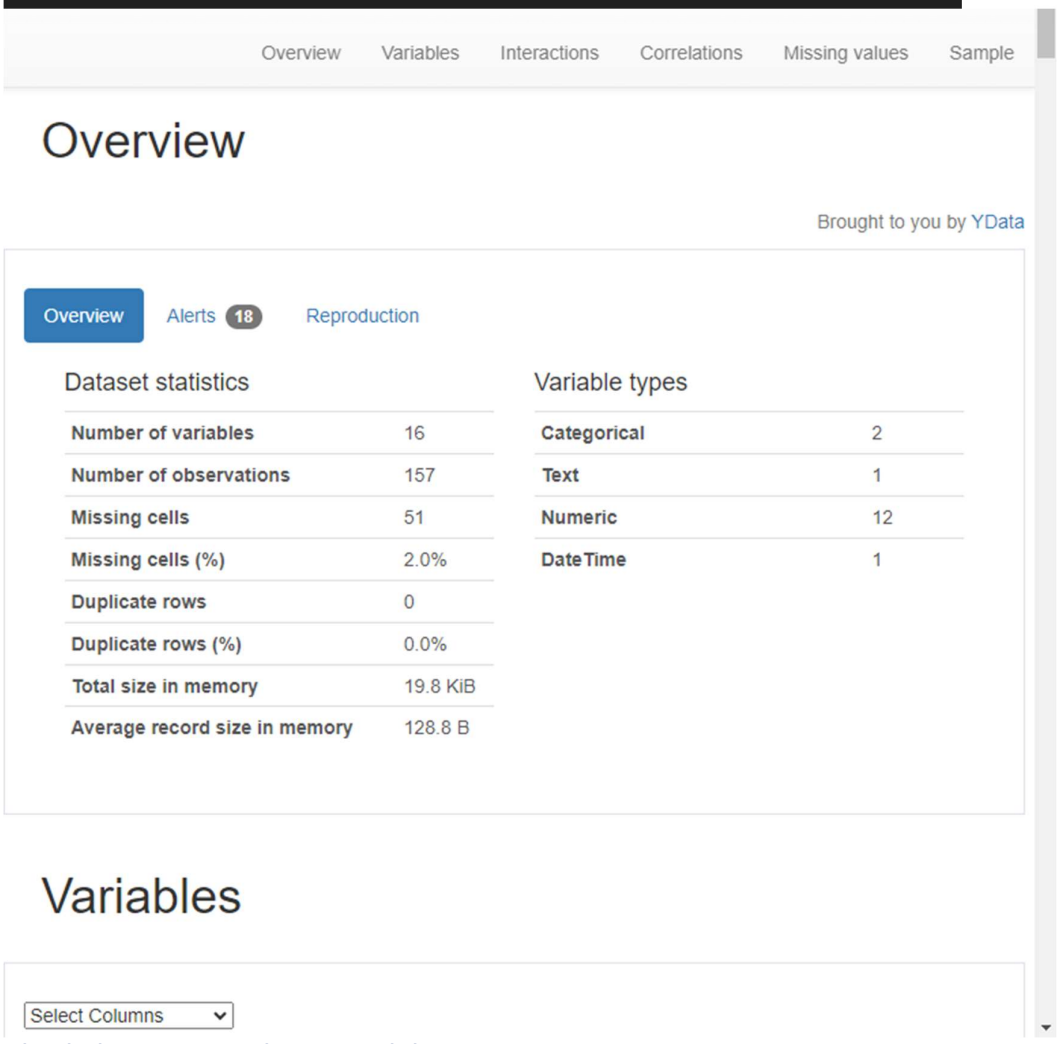
# Alternatively, you can also display the report directly
report.to_notebook_iframe()
```

```
Summarize dataset:   0%|          | 0/5 [00:00<?, ?it/s]

Generate report structure:   0%|          | 0/1 [00:00<?, ?it/s]

Render HTML:   0%|          | 0/1 [00:00<?, ?it/s]

Export report to file:   0%|          | 0/1 [00:00<?, ?it/s]
```



8. Check the missing values in mobil

```
mobil.isna().sum()
```

```
Manufacturer          0
Model                 0
Sales_in_thousands    0
__year_resale_value   36
Vehicle_type          0
Price_in_thousands    2
Engine_size           1
Horsepower            1
Wheelbase             1
Width                 1
Length                1
Curb_weight           2
Fuel_capacity         1
Fuel_efficiency       3
Latest_Launch         0
Power_perf_factor     2
dtype: int64
```

9. Check the percentage of missing value in mobil

```
# Check percentage of Missing Value,
# will be handled in next week --> Data Preparation
mobil.isnull().sum() / mobil.shape[0] * 100
```

```
Manufacturer          0.000000
Model                 0.000000
Sales_in_thousands    0.000000
__year_resale_value   22.929936
Vehicle_type          0.000000
Price_in_thousands    1.273885
Engine_size           0.636943
Horsepower            0.636943
Wheelbase             0.636943
Width                 0.636943
Length                0.636943
Curb_weight           1.273885
Fuel_capacity         0.636943
Fuel_efficiency       1.910828
Latest_Launch         0.000000
Power_perf_factor     1.273885
dtype: float64
```

10. Check for duplicates in mobil and Drop the duplicates

```
mobil.duplicated().sum()
```

```
0
```

```
mobil=mobil.drop_duplicates()
```

11. Select the numeric data types in mobil

```
# Select numeric columns
mobil = mobil.select_dtypes(include='number')
```

12. Check the shape in mobil

```
mobil.shape
```

```
(157, 12)
```

13. Fill the missing value with mean

```
mobil= mobil.fillna(mobil.mean())
```

14. Check the missing value in mobil

```
mobil.isna().sum()
```

```
Sales_in_thousands     0
__year_resale_value    0
Price_in_thousands     0
Engine_size            0
Horsepower             0
Wheelbase              0
Width                  0
Length                 0
Curb_weight            0
Fuel_capacity          0
Fuel_efficiency        0
Power_perf_factor      0
dtype: int64
```

15. See the summary of mobil

```
mobil.describe()
```

Python

	Sales_in_thousands	__year_resale_value	Price_in_thousands	Engine_size	Horsepower	Wheelbase	W
count	157.000000	157.000000	157.000000	157.000000	157.000000	157.000000	157.0
mean	52.998076	18.072975	27.390755	3.060897	185.948718	107.487179	71.1
std	68.029422	10.045282	14.259359	1.041299	56.518297	7.616772	3.4
min	0.110000	5.160000	9.235000	1.000000	55.000000	92.600000	62.6
25%	14.114000	12.545000	18.145000	2.300000	150.000000	103.000000	68.4
50%	29.450000	17.710000	23.400000	3.000000	180.000000	107.000000	70.6
75%	67.956000	18.140000	31.930000	3.500000	215.000000	112.200000	73.4
max	540.561000	67.550000	85.500000	8.000000	450.000000	138.700000	79.9

16. Importing Libraries

```
from scipy import stats
from scipy.stats import zscore
%matplotlib inline
import matplotlib.pyplot as plt
```

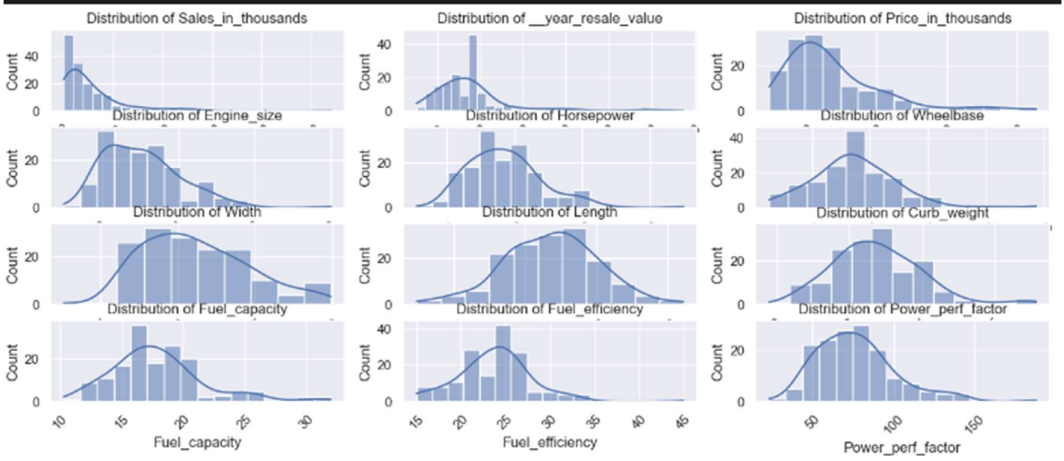
17. Visualize distribution mobil for each channel and plot on separate subplot

```
plt.figure(figsize=(15, 10))

# Iterate through each channel and plot on a separate subplot
for i, column in enumerate(mobil.columns):
    plt.subplot(7, 3, i+1)
    sns.histplot(mobil[column], kde=True)
    plt.title(f'Distribution of {column}')
    plt.xticks(rotation=45)

# Adjust layout and show the plot
#plt.tight_layout()
plt.show()
```

Pytho



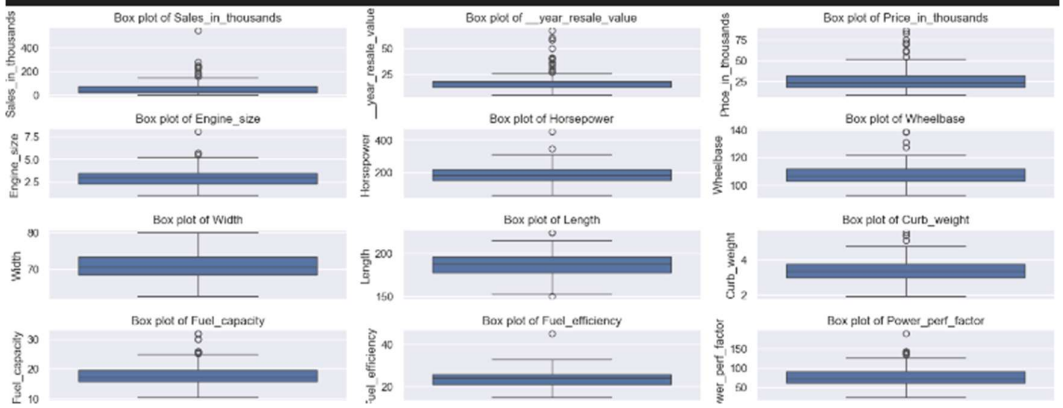
18. Visualize outliers mobil for each channel and plot on separate subplot

```
plt.figure(figsize=(15, 10))

# Iterate through each column and plot on a separate subplot
for i, column in enumerate(mobil.columns):
    plt.subplot(7, 3, i+1)
    sns.boxplot(mobil[column])
    plt.title(f'Box plot of {column}')

# Adjust layout and show the plot
plt.tight_layout()
plt.show()
```

Pytho



19. Create a function to detect outliers using Zscore for a specific column

```
# Function to detect outliers using Z-score for a specific column
def detect_outliers_z_score(mobil, threshold=3):
    z_scores = (mobil - mobil.mean()) / mobil.std()
    outliers = (np.abs(z_scores) > threshold).any(axis=1)
    return outliers
```

20. Detect outliers for each numeric column and print indices of rows containing outliers

```
# Detect outliers for each numeric column
outliers = detect_outliers_z_score(mobil[mobil.columns])

# Print indices of rows containing outliers
outlier_indices = mobil.index[outliers].tolist()
print("Index of rows with outliers:", outlier_indices)
print("Count of outliers:", len(outlier_indices))
```

```
Index of rows with outliers: [18, 26, 39, 40, 41, 42, 43, 52, 56, 74, 78, 94, 95, 99, 125, 126]
Count of outliers: 16
```

21. See the first 5 data of mobil

```
mobil.head()
```

Python

	Sales_in_thousands	__year_resale_value	Price_in_thousands	Engine_size	Horsepower	Wheelbase	Width	L
0	16.919	16.360	21.500000	1.8	140.0	101.2	67.3	
1	39.384	19.875	28.400000	3.2	225.0	108.1	70.3	
2	14.114	18.225	27.390755	3.2	225.0	106.9	70.6	
3	8.588	29.725	42.000000	3.5	210.0	114.6	71.4	
4	20.397	22.255	23.990000	1.8	150.0	102.6	68.2	

22. Remove outliers based on Zscore for a specific column

```
# Remove outliers based on Z-score for a specific column
def remove_outliers_z_score_column(data_column, threshold=3):
    z_scores = zscore(data_column)
    filtered_data = data_column[np.abs(z_scores) <= threshold]
    return filtered_data
```

23. Save the result to dataframe 'updated_data'

```
updated_data = remove_outliers_z_score_column(mobil)
```

24. See the first 15 data of updated_data

```
updated_data.head(15)
```

Python

	Sales_in_thousands	__year_resale_value	Price_in_thousands	Engine_size	Horsepower	Wheelbase	Width
0	16.919	16.360000	21.500000	1.8	140.0	101.2	67.3
1	39.384	19.875000	28.400000	3.2	225.0	108.1	70.3
2	14.114	18.225000	27.390755	3.2	225.0	106.9	70.6
3	8.588	29.725000	42.000000	3.5	210.0	114.6	71.4
4	20.397	22.255000	23.990000	1.8	150.0	102.6	68.2
5	18.780	23.555000	33.950000	2.8	200.0	108.7	76.1
6	1.380	39.000000	62.000000	4.2	310.0	113.0	74.0
7	19.747	18.072975	26.990000	2.5	170.0	107.3	68.4
8	9.231	28.675000	33.400000	2.8	193.0	107.3	68.5
9	17.527	36.125000	38.900000	2.8	193.0	111.4	70.9
10	91.561	12.475000	21.975000	3.1	175.0	109.0	72.7
11	39.350	13.740000	25.300000	3.8	240.0	109.0	72.7
12	27.851	20.190000	31.965000	3.8	205.0	113.8	74.7
13	83.257	13.360000	27.885000	3.8	205.0	112.2	73.5
14	63.729	22.525000	39.895000	4.6	275.0	115.3	74.5

25. Detect outliers for each numeric column and print indices of rows containing outliers for the updated_data.

```
# Detect outliers for each numeric column
outliers = detect_outliers_z_score(updated_data[updated_data.columns])

# Print indices of rows containing outliers
outlier_indices = updated_data.index[outliers].tolist()
print("Index of rows with outliers:", outlier_indices)
print("Count of outliers:", len(outlier_indices))
```

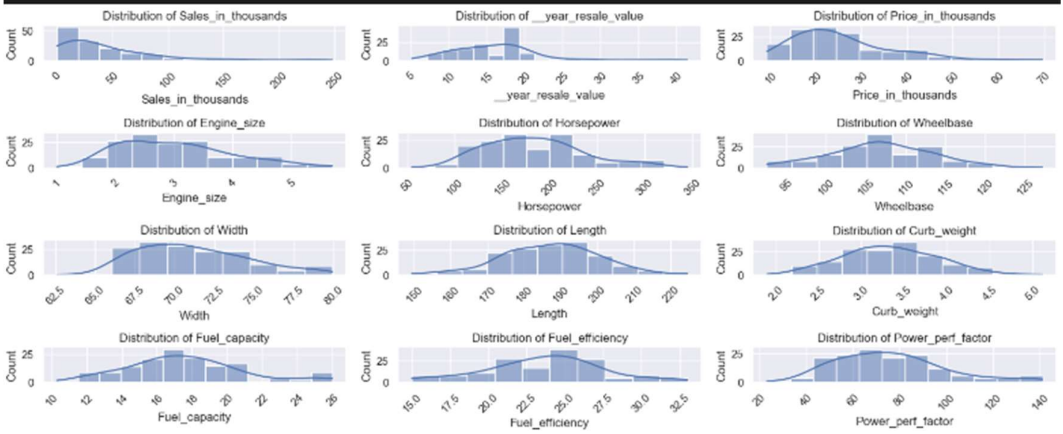
```
Index of rows with outliers: [6, 24, 39, 40, 42, 49, 55, 58, 73, 93, 94, 124, 137, 144]
Count of outliers: 14
```

26. Visualize distribution check mobil for each channel and plot on separate subplot

```python
plt.figure(figsize=(15, 10))

# Iterate through each channel and plot on a separate subplot
for i, column in enumerate(updated_data.columns):
    plt.subplot(7, 3, i+1)
    sns.histplot(updated_data[column], kde=True)
    plt.title(f'Distribution of {column}')
    plt.xticks(rotation=45)

# Adjust layout and show the plot
plt.tight_layout()
plt.show()
```

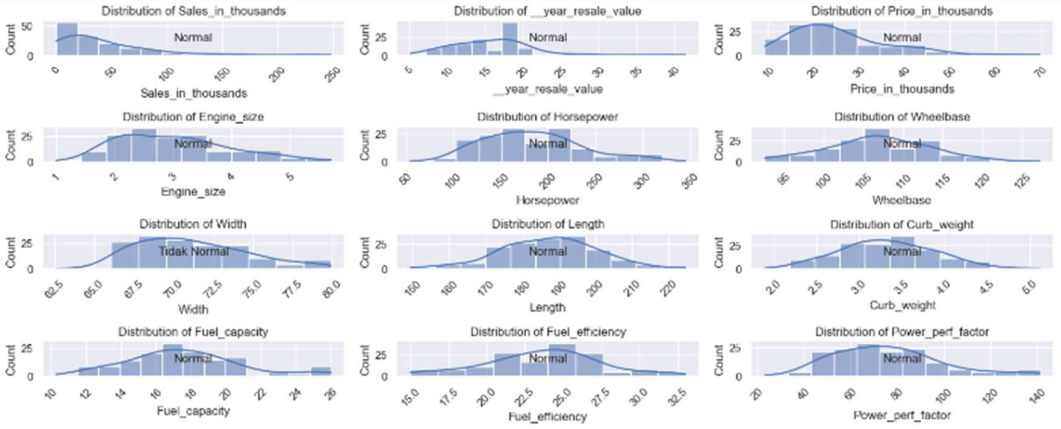


```python
plt.figure(figsize=(15, 10))

# Iterate through each column and plot on a separate subplot
for i, column in enumerate(updated_data.columns):
    plt.subplot(7, 3, i+1)
    sns.histplot(updated_data[column], kde=True)
    plt.title(f'Distribution of {column}')
    plt.xticks(rotation=45)

    # Add additional analysis to detect distribution type
    # Check for normal distribution
    k2, p = stats.normaltest(updated_data[column])
    if p < 0.05:
        plt.text(0.5, 0.5, "Tidak Normal",
                 horizontalalignment='center',
                 verticalalignment='center', ransform=plt.gca().transAxes)
    else:
        plt.text(0.5, 0.5, "Normal", horizontalalignment='center',
                 verticalalignment='center', transform=plt.gca().transAxes)

# Adjust layout and show the plot
plt.tight_layout()
plt.show()
```



Binning technique to remove Skewness

A binning technique is a method used to group or divide numerical data into multiple intervals (called "bins" or "baskets") with the aim of reducing the skewness of the data distribution.

Skewness is a measure of the asymmetry of data distribution, where unsymmetrical data can affect statistical analysis and machine learning models.

27. Calculate Skewness

```python
# Calculate skewness
skewness = updated_data['Sales_in_thousands'].skew()
print("Skewness:", skewness)
```

```
Skewness: 1.9608492073723116
```

28. Apply binning to Feature 3 and Feature 4

```python
num_bins = 50

# Apply binning to Feature 3 and Feature 4
updated_data['Sales_in_thousands_binned'] = pd.qcut(updated_data['Sales_in_thousands'],
                                                    q=num_bins, labels=False)
```

29. Display the first 5 data of binned features

```
# Display the first few rows of the dataset with binned features
updated_data.head()
```

Python

	Sales_in_thousands	__year_resale_value	Price_in_thousands	Engine_size	Horsepower	Wheelbase	Width	L
0	16.919	16.360	21.500000	1.8	140.0	101.2	67.3	
1	39.384	19.875	28.400000	3.2	225.0	108.1	70.3	
2	14.114	18.225	27.390755	3.2	225.0	106.9	70.6	
3	8.588	29.725	42.000000	3.5	210.0	114.6	71.4	
4	20.397	22.255	23.990000	1.8	150.0	102.6	68.2	

30. See the info of updated_data

```
updated_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 157 entries, 0 to 156
Data columns (total 13 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Sales_in_thousands        155 non-null    float64
 1   __year_resale_value       152 non-null    float64
 2   Price_in_thousands        153 non-null    float64
 3   Engine_size               156 non-null    float64
 4   Horsepower                156 non-null    float64
 5   Wheelbase                 154 non-null    float64
 6   Width                     157 non-null    float64
 7   Length                    157 non-null    float64
 8   Curb_weight               154 non-null    float64
 9   Fuel_capacity             153 non-null    float64
 10  Fuel_efficiency           156 non-null    float64
 11  Power_perf_factor         156 non-null    float64
 12  Sales_in_thousands_binned  155 non-null    float64
dtypes: float64(13)
memory usage: 16.1 KB
```

31. Drop 'Sales_in_thousands' from updated_data

```
updated_data.drop(['Sales_in_thousands'],axis=1,inplace=True)
```

32. Rename 'Sales_in_thousands_binned' to 'Sales_in_thousands'

```
updated_data.rename(columns={"Sales_in_thousands_binned": "Sales_in_thousands"},inplace=True)
```

33. See the updated info of updated_data

```
updated_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 157 entries, 0 to 156
Data columns (total 12 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   __year_resale_value  152 non-null    float64
 1   Price_in_thousands   153 non-null    float64
 2   Engine_size          156 non-null    float64
 3   Horsepower           156 non-null    float64
 4   Wheelbase            154 non-null    float64
 5   Width                157 non-null    float64
 6   Length               157 non-null    float64
 7   Curb_weight          154 non-null    float64
 8   Fuel_capacity        153 non-null    float64
 9   Fuel_efficiency      156 non-null    float64
 10  Power_perf_factor    156 non-null    float64
 11  Sales_in_thousands   155 non-null    float64
dtypes: float64(12)
memory usage: 14.8 KB
```

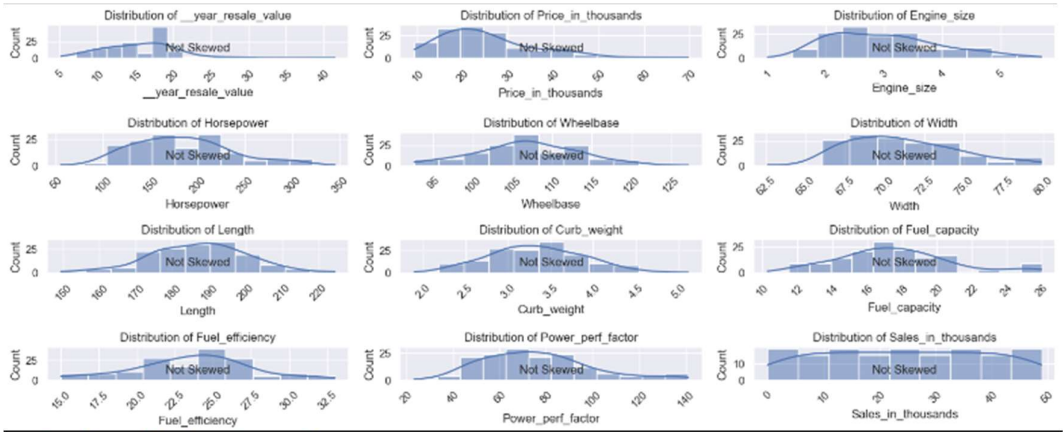
34. Confirm if Skewness is removed or not by visualizing

```
plt.figure(figsize=(15, 10))

# Iterate through each column and plot on a separate subplot
for i, column in enumerate(updated_data.columns):
    plt.subplot(7, 3, i+1)
    sns.histplot(updated_data[column], kde=True)
    plt.title(f'Distribution of {column}')
    plt.xticks(rotation=45)

    # Check for skewness
    skewness = stats.skew(updated_data[column])
    if skewness < -1 or skewness > 1:
        plt.text(0.5, 0.3, f"Skewed ({skewness:.2f})",
                 horizontalalignment='center',
                 verticalalignment='center', transform=plt.gca().transAxes)
    else:
        plt.text(0.5, 0.3, f"Not Skewed",
                 horizontalalignment='center',
                 verticalalignment='center', transform=plt.gca().transAxes)

# Adjust layout and show the plot
plt.tight_layout()
plt.show()
```

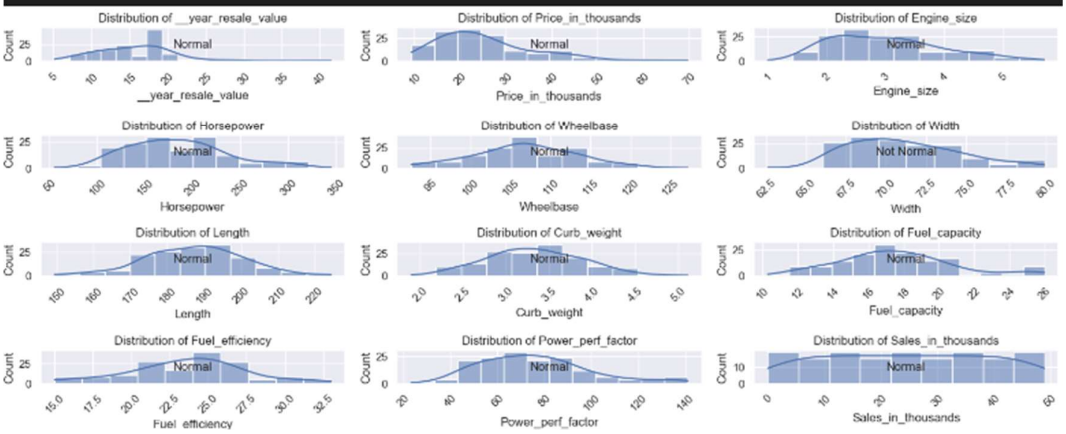



```python
plt.figure(figsize=(15, 10))

# Iterate through each column and plot on a separate subplot
for i, column in enumerate(updated_data.columns):
    plt.subplot(7, 3, i+1)
    sns.histplot(updated_data[column], kde=True)
    plt.title(f'Distribution of {column}')
    plt.xticks(rotation=45)

    # Add additional analysis to detect distribution type
    # Check for normal distribution
    k2, p = stats.normaltest(updated_data[column])
    if p < 0.05:
        plt.text(0.5, 0.5, "Not Normal",
                 horizontalalignment='center',
                 verticalalignment='center', transform=plt.gca().transAxes)
    else:
        plt.text(0.5, 0.5, "Normal",
                 horizontalalignment='center',
                 verticalalignment='center', transform=plt.gca().transAxes)

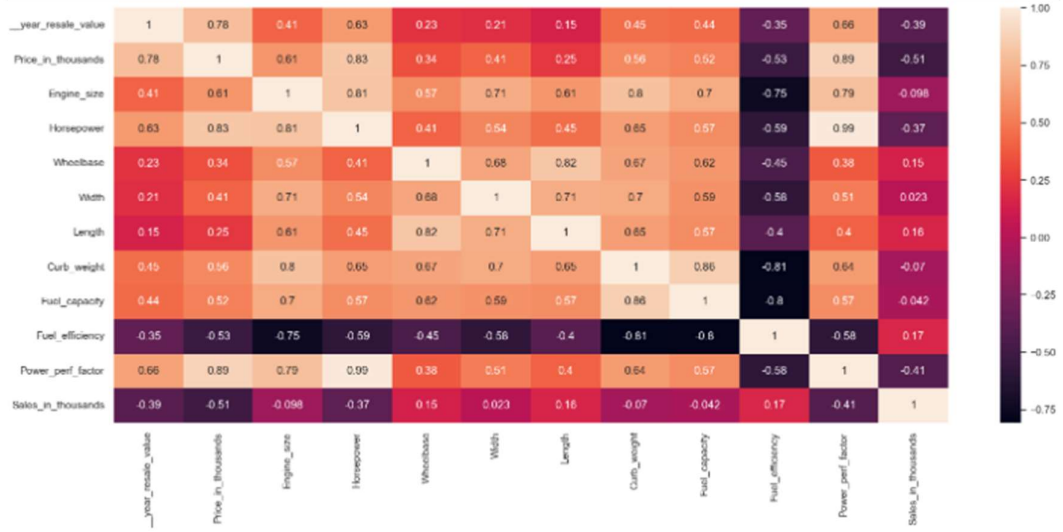
# Adjust layout and show the plot
plt.tight_layout()
plt.show()
```



35. Visualize the Correlation of the updated_data using heatmap

```
# Correlation using heatmap
#this data will not capable for correlation due to categorical data
sns.heatmap(updated_data.corr(), annot=True)
```

```
<Axes: >
```



36. Visualize the pairplot for the updated_data

```
sns.pairplot(updated_data, hue='Sales_in_thousands')
```

```
<seaborn.axisgrid.PairGrid at 0x270701ed1c0>
```



37. Add row 'id' for the updated_data

```
updated_data.insert(0, 'id', range(1, 1 + len(updated_data)))
updated_data.head()
```

	id	__year_resale_value	Price_in_thousands	Engine_size	Horsepower	Wheelbase	Width	Length	Curb_we
0	1	16.360	21.500000	1.8	140.0	101.2	67.3	172.4	2
1	2	19.875	28.400000	3.2	225.0	108.1	70.3	192.9	3
2	3	18.225	27.390755	3.2	225.0	106.9	70.6	192.0	3
3	4	29.725	42.000000	3.5	210.0	114.6	71.4	196.6	3
4	5	22.255	23.990000	1.8	150.0	102.6	68.2	178.0	2

38. Group updated_data by 'Price_in_thousands' with unique 'id'

```
# Ecample of imbalance data for each class

updated_data.groupby('Price_in_thousands')['id'].nunique()
```

```
Price_in_thousands
9.235     1
9.699     1
10.685    1
11.528    1
11.799    1
         ..
54.005    1
60.105    1
62.000    1
69.700    1
69.725    1
Name: id, Length: 149, dtype: int64
```

39. Calculate the skewness for each column in updated_data while skipping any NaN values.

```
updated_data.skew(axis = 0, skipna = True)
```

```
id                    0.000000
__year_resale_value   1.543072
Price_in_thousands    1.336437
Engine_size           0.618043
Horsepower            0.525632
Wheelbase             0.092676
Width                 0.485380
Length               -0.059232
Curb_weight           0.218509
Fuel_capacity         0.538622
Fuel_efficiency      -0.032788
Power_perf_factor     0.711793
Sales_in_thousands    0.000672
dtype: float64
```

The updated_data.skew(axis=0, skipna=True) command is used to calculate the skewness of each column in the updated_data DataFrame. Skewness is a statistical measure that indicates the extent to which the data distribution is skewed to the right (positive) or to the left (negative).

A normal distribution has a skewness close to 0, while a skewed distribution has a larger skewness value (positive or negative).